



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and
Information Technology

EBB 320

CONTROL SYSTEMS

PRACTICAL 1: HARDWARE IMPLEMENTATION AND SYSTEM IDENTIFICATION

Name and Surname	Student Number	Signature	% Contribution
Hanré Coetzer	21507334		33
Cornelis Claasen	24704611		33
B.J. Mncube	23700816		33

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

October 2, 2026

Contents

1	Theoretical Background	2
2	Software and Hardware Design Implementation	3
2.1	MCU Code	3
2.2	Matlab Code	5
2.3	Hardware Development	7
2.3.1	Circuit Design	7
2.3.2	Implementation	7
2.3.3	Temperature Calibration Equation	7
3	Results	9
3.1	Step Test Data Acquired from System	9
3.2	Derived FOPDT transfer functions	10
3.2.1	Process Reaction Curve Method	10
3.2.2	Variables Derived from Process Reaction Curves	13
3.2.3	Summary of Process Reaction Curve Results	15
3.3	Matlab fitting	15
3.3.1	Summary of Validation Data	19
4	Discussion	20
4.1	Model validation	20
4.1.1	Accuracy metric	20
4.1.2	Model accuracy and Transfer function comparisons	20
5	Code	22
5.1	Matlab Code	22
5.2	Model Fit Code	36
5.3	MCU Code	38

1 Theoretical Background

The accuracy of the estimated transfer function models was evaluated using the Normalized Root Mean Square Error (NRMSE) fit percentage. This metric quantifies how well the simulated model response matches the measured physical data, and is defined as:

$$\text{Fit (\%)} = 100 \times \left(1 - \frac{\|y - \hat{y}\|}{\|y - \bar{y}\|} \right) \quad (1)$$

where y is the measured output data, $\hat{y}$ is the simulated model output, $\bar{y}$ is the mean of the measured output data, and $\|\cdot\|$ denotes the Euclidean norm.

2 Software and Hardware Design Implementation

2.1 MCU Code

In order to allow for easier future interaction, a command system was introduced. The MCU receives commands over UART as a string with the format:

"<command>, <value>"

These commands are interpreted by the MCU, changing the appropriate values.

Table 1: MCU Commands

Command	Value Changed
0	Change P
1	Change I
2	Change D
3	Change Setpoint
4	Change Disturbance Value
5	Change Duty Cycle
6	Change PWM Output
7	Change Start Flag

If the start variable is set, then the sampling process begins. First a timer is started, the PWM output flags variable is then checked to see which PWM signals must be set, and finally the samples are taken.

The samples are converted to temperatures using the temperature calibration equation. A running average of 200 samples is used to filter high frequency noise. The timer started at the beginning of the sampling process is then checked to see if the sampling period has passed, and if not, it waits for the remaining time. The processed samples are again sent over UART as a string with the format:

"<Temp1>, <Temp2>, <PWM1>, <PWM2>, <Time>, <Message>"

The <Message> parameter is used to signal error data. Currently, the only error message implemented is for if the sampling process takes longer than the set sampling period. (Currently 200 Hz)

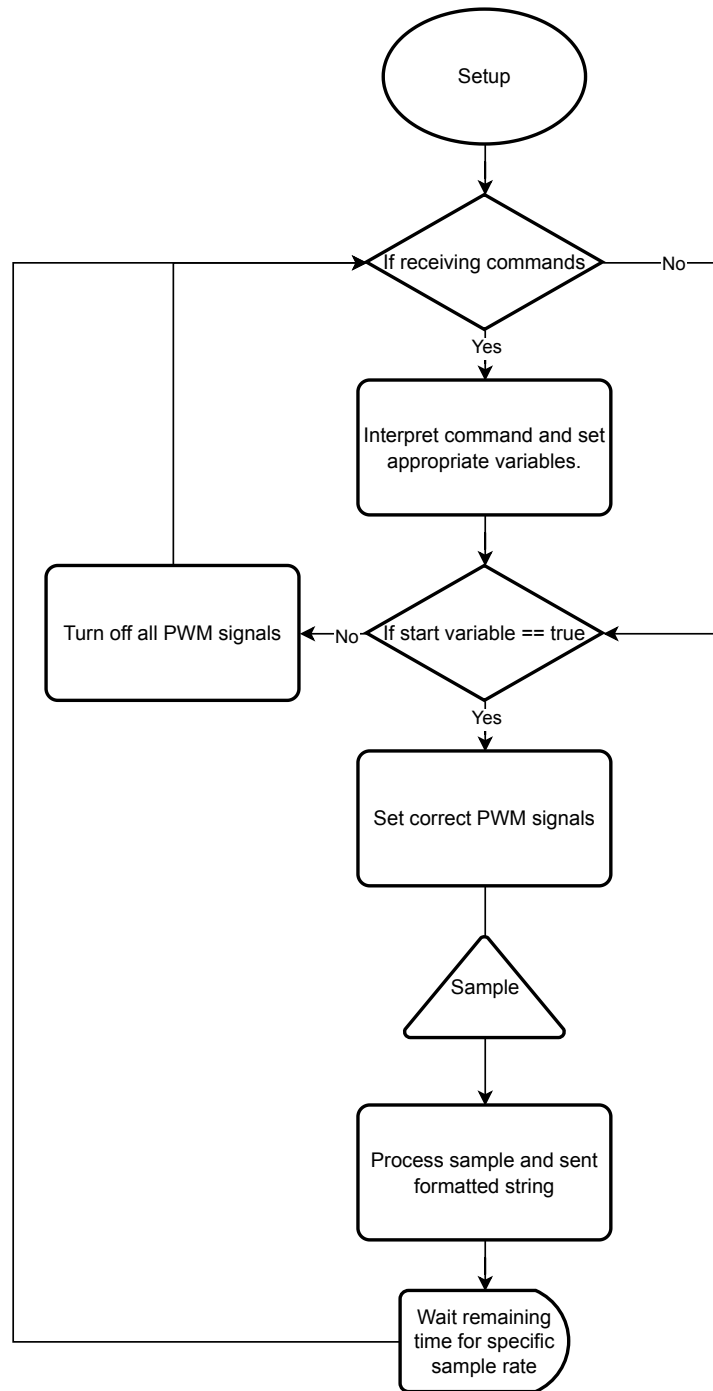


Figure 1: Flow Diagram of MCU Code.

2.2 Matlab Code

The Matlab code was written into a Matlab app to make it easier to operate the system. The app waits for the start button to be pressed, after which it sends the commands from table 1 to the MCU to update all the parameters set.

It then sets up the plotting variables and a variable called "capturing", which is used to continue the plotting process in a while loop, until the stop button is pressed. The loop continually checks if it has received data, and if it has, it interprets the formatted string send by the MCU. It plots the received data, checks the error messages received, and displays information based on the received message.

Before the loop starts a .csv file is accessed (or created if it did not exist beforehand), and the formatted string received from the MCU is saved directly to it.

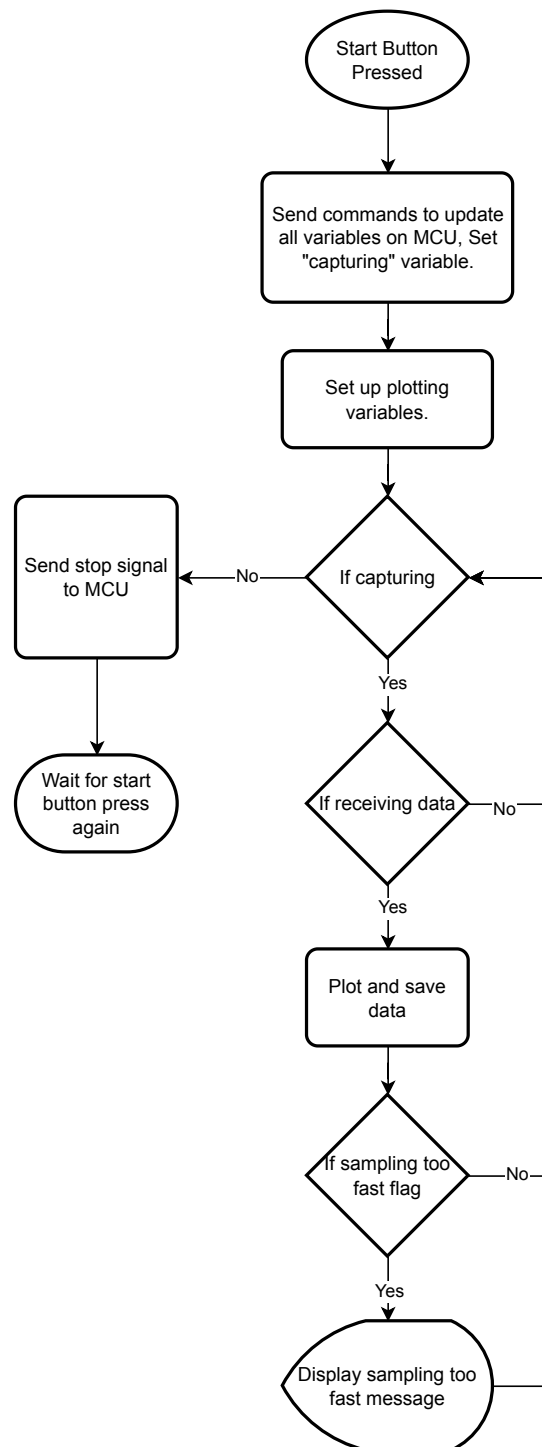


Figure 2: Flow Diagram of Matlab Code.

2.3 Hardware Development

2.3.1 Circuit Design

The physical system consists of two identical channels, each built around a TIP31C power transistor and an MCP9700 linear temperature sensor. In this application the transistor does not operate as a signal amplifier; instead, it functions as a PWM-controlled current driver. The base of each transistor is switched by a PWM signal generated on the ESP32, which sets the average current delivered from the 5 V supply through the transistor into the heat sink. This produces a direct causal chain from the controllable input to the measured output:

PWM duty cycle $\rightarrow$ transistor conduction current $\rightarrow$ power dissipated as heat $\rightarrow$
heat sink temperature $\rightarrow$ sensor output voltage

This chain constitutes the physical plant that is later identified as a transfer function in the third section of this report.

Because the ESP32 operates its GPIO logic at 3.3 V, the base resistor values were selected according to the 3.3 V column of Table 1 in the practical guide[1]: $R_1 = 270 \Omega$ and $R_2 = 560 \Omega$. These resistors were chosen to limit the base current drawn from the ESP32 GPIO pin to a safe level while still supplying enough current to fully switch the TIP31C on, given the fixed 5 V bench-supply rail feeding the collector.

The MCP9700 temperature sensor was selected as it requires no external signal-conditioning circuitry; its output pin can be connected directly to an ESP32 ADC input. The sensor was mounted in direct physical contact with the TIP31C package (and, by extension, the heat sink) to ensure accurate thermal coupling between the heat source and the measurement point.

2.3.2 Implementation

The circuit was constructed on a veroboard following the layout described in the practical guide, with both transistor/sensor pairs mounted such that their heat sinks were in direct contact with one another, ensuring consistent thermal cross-coupling between channels for later system identification of the off-diagonal transfer functions (G_{12} , G_{21}). The 5 V supply rail was provided by a bench power supply set to its maximum allowable current limit, as an ESP32 GPIO pin cannot source sufficient current to drive the transistor directly. The ESP32 shared a common ground with the transistor circuit's power supply to ensure a consistent reference voltage across the system.

2.3.3 Temperature Calibration Equation

The MCP9700 does not output a digital temperature value; it is a purely analogue device whose output voltage varies linearly with the temperature. Microchip characterises this relationship in the datasheet as a first-order (linear) transfer function [2]:

$$V_{OUT} = T_C \cdot T_A + V_{0^\circ C} \quad (2)$$

where V_{OUT} is the sensor's output voltage, T_A is the ambient temperature, T_C is the temperature coefficient (the slope of the line, in V/°C), and $V_{0^\circ C}$ is the sensor's

output voltage at 0°C (the y-intercept). This is a standard linear equation of the form $y = mx + c$.

For the MCP9700/9700A/9700B family, the datasheet’s DC Electrical Characteristics table specifies:

$$T_C = 10.0 \text{ mV}/^\circ\text{C}, \quad V_{0^\circ\text{C}} = 500 \text{ mV} \quad (3)$$

Since the ADC measures a voltage and the objective is to recover a temperature, Equation (2) is rearranged to make T_A the subject:

$$T_A = \frac{V_{OUT} - V_{0^\circ\text{C}}}{T_C} \quad (4)$$

Substituting the datasheet constants, with the voltage values expressed in volts ($V_{0^\circ\text{C}} = 0.5 \text{ V}$, $T_C = 0.01 \text{ V}/^\circ\text{C}$):

$$T_A(^\circ\text{C}) = \frac{V_{ADC} - 0.5}{0.01} \quad (5)$$

Equation (5) is identical to Equation 1 given in the practical guide. The substitution of V_{ADC} for V_{OUT} reflects the physical circuit: since the sensor output is connected directly to the ESP32’s ADC input with no intermediate signal conditioning, the voltage sampled by the ADC is equal to the sensor’s output voltage.

The sensor required no additional physical calibration step; it is a factory-characterised property of the device. The datasheet specifies a typical accuracy of $\pm 4^\circ\text{C}$ over the 0–70°C range for the MCP9700, which represents an inherent source of measurement uncertainty carried into the system identification results discussed later in this report.

3 Results

3.1 Step Test Data Acquired from System

System Identification Process

System identification (SID) is a data-driven modelling approach: rather than deriving a transfer function from the underlying physics of the transistor–heat-sink–sensor system (a “white box” approach), the model is instead inferred directly from measured input-output data (a “black box” approach). This is well suited to this practical, since the thermal dynamics of the physical setup (heat generation in the transistor, conduction through the heat sink, convective losses to the surrounding air) are not trivial to derive analytically to the accuracy required.

$$G_{ij} = \frac{K_{ij}}{\tau_{ij}s + 1} e^{-\theta_{ij}s}, \quad (6)$$

The SID process followed in this practical can be summarised as: (1) experiment and data collection, in which a known input is applied to the system and the resulting output is recorded; (2) selection of a model structure, fixed for this practical as the first-order-plus-dead-time (FOPDT) form given in Equation (6); (3) fitting the model structure to the collected data, performed using two independent methods – the process reaction curve method and MATLAB’s transfer function estimation tools; and (4) validating the fitted model against data that played no part in the fitting process.

Testing vs. Validation

Test data is the input-output data actually used to calculate the model parameters, in this case, the step-response data used to extract θ , τ and K for each element of the transfer function matrix. Validation data, by contrast, is an entirely separate dataset, never used during fitting, that is used afterwards to assess how accurately the fitted model predicts the behaviour of the real system.

This distinction matters because a model’s accuracy on the data it was fitted to is not a reliable indicator of its real-world performance: since the parameters were explicitly chosen to match that data, evaluating accuracy on it would overstate the quality of the fit. Validation on independent data gives a fair, unbiased test of the model’s ability to generalise to inputs it has not seen.

Division of Test and Validation Data

For each input channel, two independent step-test datasets were acquired. The test (fitting) dataset was recorded with the relevant PWM channel stepped to approximately 97–100% duty cycle, and was used exclusively to derive the FOPDT parameters (θ , τ , K) via both the process reaction curve method and MATLAB transfer function fitting. The validation dataset was recorded in a separate acquisition session and steps the same channel to a substantially different operating point of 50% duty cycle, before being held for the remainder of the recording.

This separation satisfies the independence requirement in two respects. First, the validation data was captured in a distinct recording session from the test data, so no samples are shared between the two sets. Second, and more importantly, the validation step targets a different duty cycle than the one used for fitting; since the fitted models were never exposed to system behaviour at this operating point, their performance on the validation data is a genuine test of how well the linear FOPDT approximation generalises across the input range, rather than a measure of how well the model memorises the specific step it was fitted to.

3.2 Derived FOPDT transfer functions

The transfer functions of the system were derived using two different methods, namely the process reaction curve method and transfer function fitting using MATLAB. Each method yielded a MIMO transfer function matrix with the following mathematical description:

$$\begin{bmatrix} Y_1(s) \\ Y_2(s) \end{bmatrix} = \begin{bmatrix} G_{11}(s) & G_{12}(s) \\ G_{21}(s) & G_{22}(s) \end{bmatrix} \begin{bmatrix} U_1(s) \\ U_2(s) \end{bmatrix} \quad (7)$$

The following table contains these two matrices.

Table 2: Comparison of Transfer Functions: Transfer Function Fitting vs. Process Reaction Curve

Thermal Pathway	Transfer Function Fitting (tfest)	Process Reaction Curve
$G_{11}(s)$ (PWM 1 $\rightarrow$ T1)	$\frac{-0.1974s+0.05300}{s^2+0.4582s+0.0034}$	$\frac{15.376}{158.9s+1}e^{-4.0s}$
$G_{12}(s)$ (PWM 2 $\rightarrow$ T1)	$\frac{-0.0156s+0.0010}{s^2+0.0300s+0.0002}$	$\frac{6.724}{198.0s+1}e^{-29.0s}$
$G_{21}(s)$ (PWM 1 $\rightarrow$ T2)	$\frac{-0.0097s+0.0016}{s^2+0.0268s+0.0002}$	$\frac{10.430}{328.5s+1}e^{-18.7s}$
$G_{22}(s)$ (PWM 2 $\rightarrow$ T2)	$\frac{-0.0346s+0.0234}{s^2+0.3998s+0.0024}$	$\frac{9.712}{198.0s+1}e^{-1.5s}$

Table 2 compares the transfer functions that were derived using the two different methods. It is noted that the transfer function fitting method was not restricted to a first order system unlike the Process Reaction Curve method.

3.2.1 Process Reaction Curve Method

Figures 3 and 4 show the reaction curves obtained from the PWM1 and PWM2 step tests respectively. The tangent line at the point of maximum slope is drawn for each channel, with markers indicating t_1 (tangent intersects y_0) and t_2 (tangent intersects y_∞).

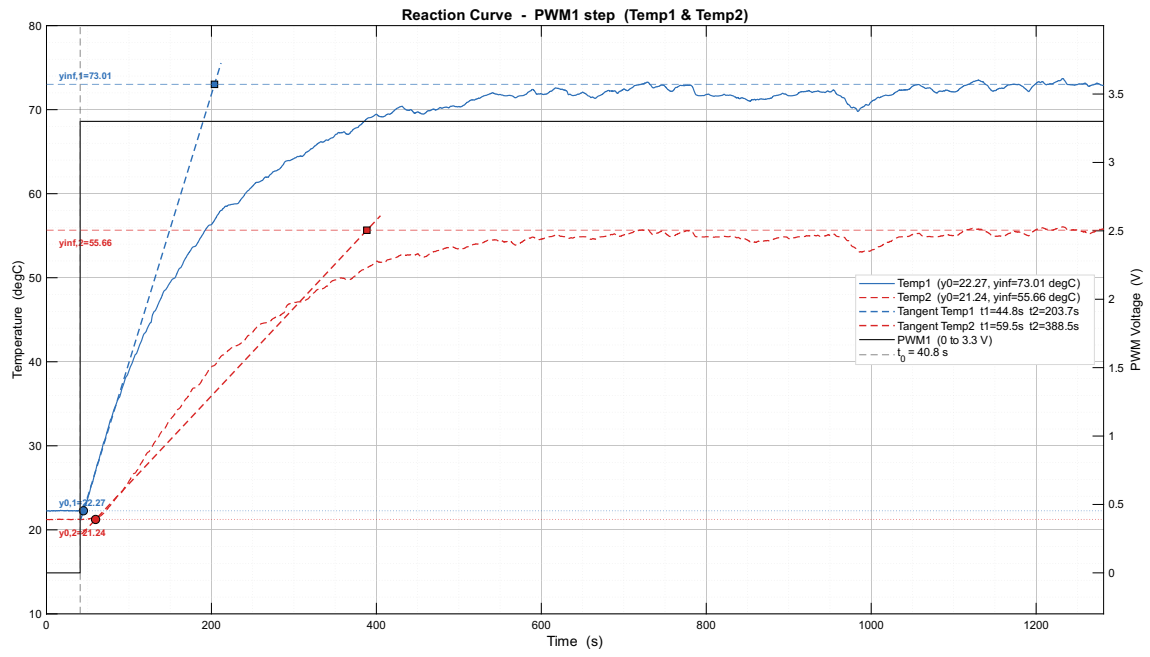


Figure 3: Reaction curve for PWM1 step input showing Temp1 and Temp2 responses.

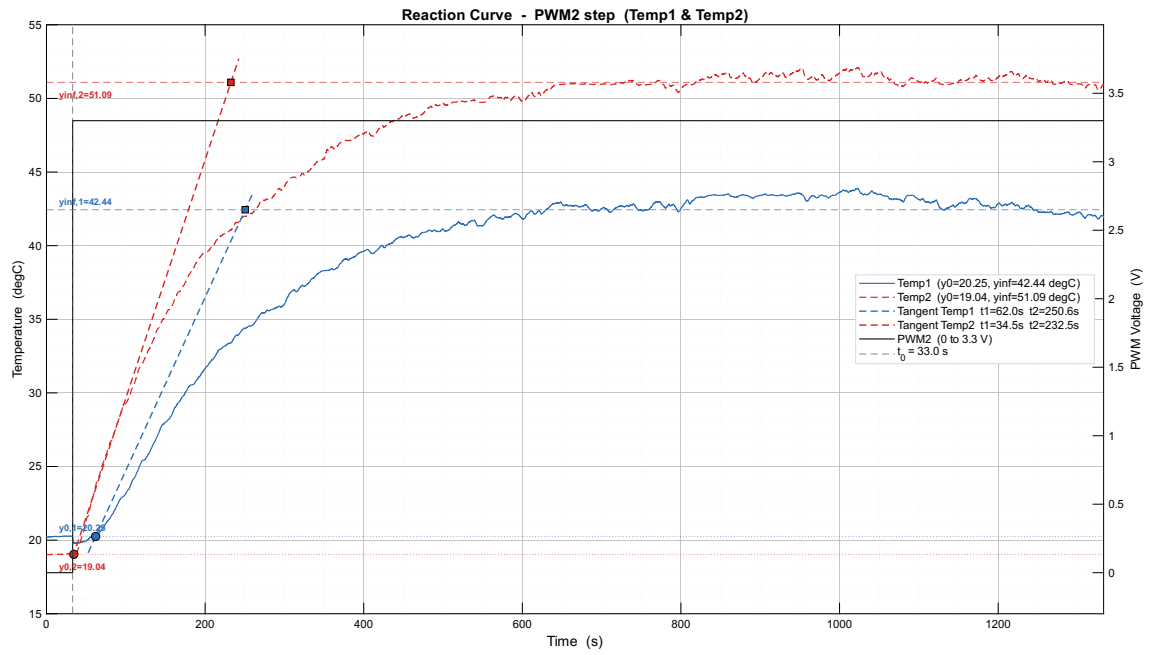


Figure 4: Reaction curve for PWM2 step input showing Temp1 and Temp2 responses.

Figures 5 and 6 showcases the models accuracy using test data obtained by applying a 50% duty cycle PWM

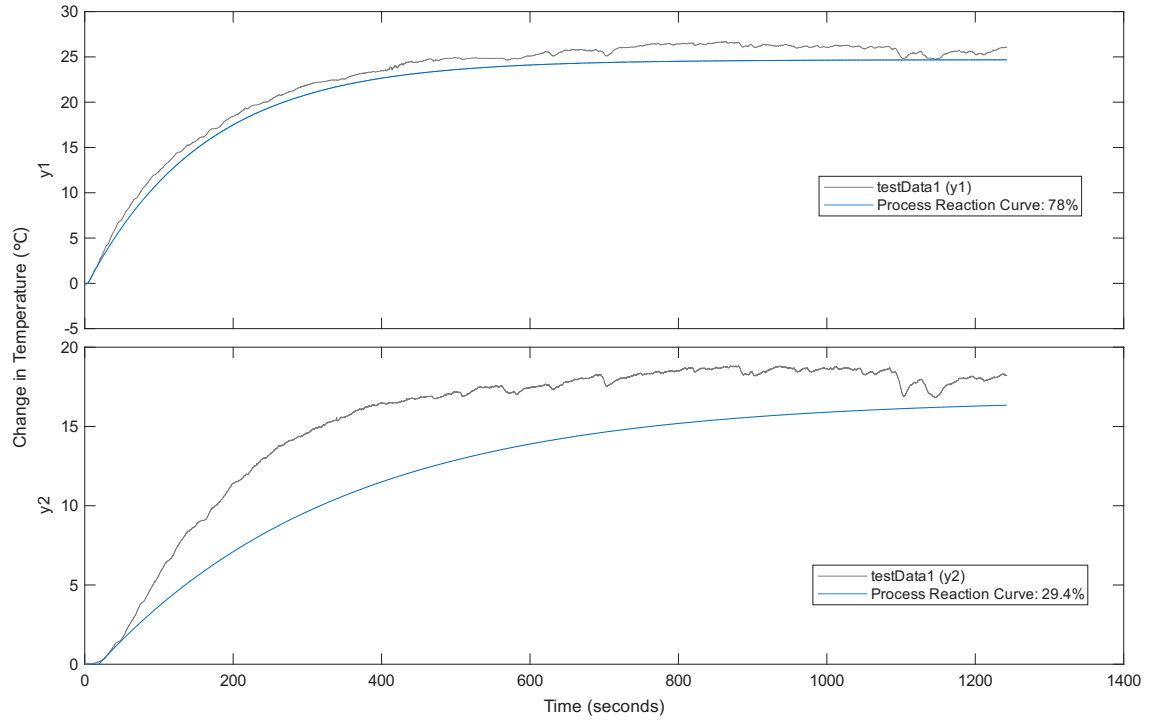


Figure 5: Simulated Process reaction curve model G_{11} (top) and G_{21} (bottom) using a 50% duty cycle as input.

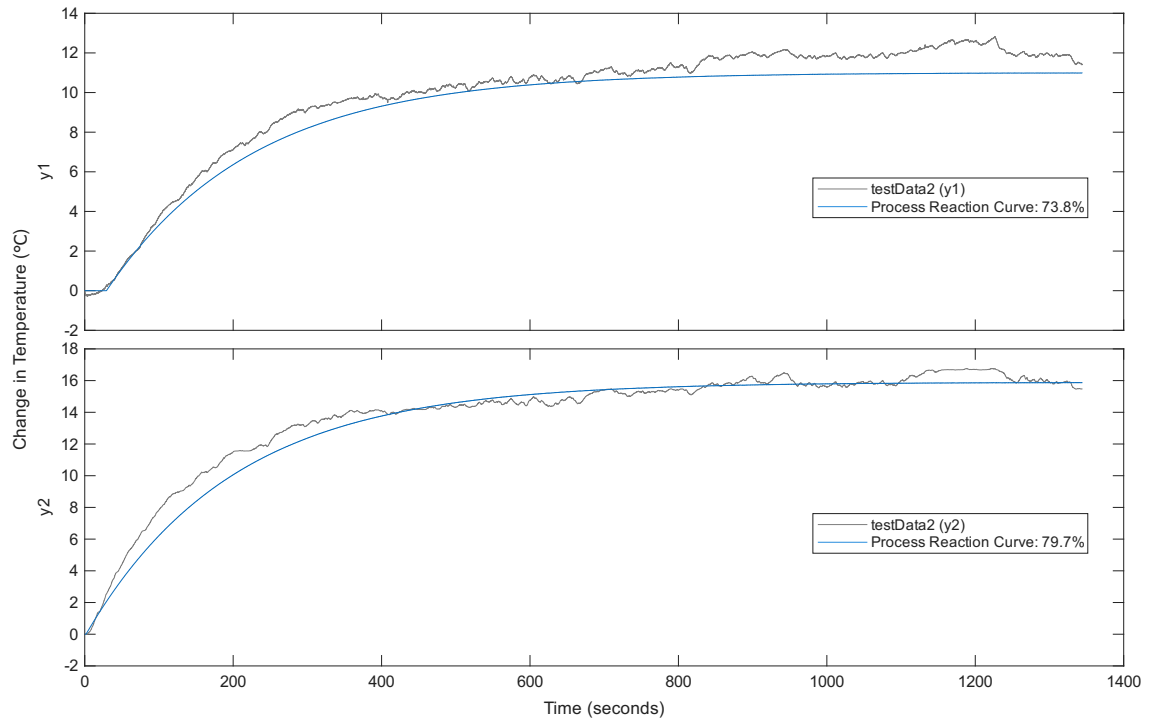


Figure 6: Simulated Process reaction curve model G_{22} (bottom) and G_{12} (top) using a 50% duty cycle as input.

3.2.2 Variables Derived from Process Reaction Curves

The following variables were derived from the reaction curves using the tangent line method. All derivations were performed using the plots obtained in MATLAB.

Table 3: Variables for G_{11}

Variable	Value
t_0	40.8 s
t_1	44.8 s
t_2	203.7 s
Δu	3.3 V
Δy	$73.01 - 22.27 = 50.74$ °C
θ	$44.8 - 40.8 = 4.0$ s
τ	$203.7 - 44.8 = 158.9$ s
K	$\frac{50.74}{3.3} = 15.376$ °C/V

Table 4: Variables for G_{21}

Variable	Value
t_0	40.8 s
t_1	59.5 s
t_2	388.0 s
Δu	3.3 V
Δy	$55.66 - 21.24 = 34.42$ °C
θ	$59.5 - 40.8 = 18.7$ s
τ	$388.0 - 59.5 = 328.5$ s
K	$\frac{34.42}{3.3} = 10.430$ °C/V

Table 5: Variables for G_{12}

Variable	Value
t_0	33.0 s
t_1	62.0 s
t_2	260.0 s
Δu	3.3 V
Δy	$42.44 - 20.25 = 22.19$ °C
θ	$62.0 - 33.0 = 29.0$ s
τ	$260.0 - 62.0 = 198.0$ s
K	$\frac{22.19}{3.3} = 6.724$ °C/V

Table 6: Variables for G_{22}

Variable	Value
t_0	33.0 s
t_1	34.5 s
t_2	232.5 s
Δu	3.3 V
Δy	$51.09 - 19.04 = 32.05$ °C
θ	$34.5 - 33.0 = 1.5$ s
τ	$232.5 - 34.5 = 198.0$ s
K	$\frac{32.05}{3.3} = 9.712$ °C/V

The resulting FOPDT transfer functions are:

$$G_{11}(s) = \frac{15.376 e^{-4.0s}}{158.9s + 1} \quad (8)$$

$$G_{21}(s) = \frac{10.430 e^{-18.7s}}{328.5s + 1} \quad (9)$$

$$G_{12}(s) = \frac{6.724 e^{-29.0s}}{198.0s + 1} \quad (10)$$

$$G_{22}(s) = \frac{9.712 e^{-1.5s}}{198.0s + 1} \quad (11)$$

3.2.3 Summary of Process Reaction Curve Results

Table 7: Process Reaction Curve results summary

Transfer Function	K (°C/V)	τ (s)	θ (s)	Method
$G_{11}(s)$	15.376	158.9	4.0	PRC
$G_{21}(s)$	10.430	328.5	18.7	PRC
$G_{12}(s)$	6.724	198.0	29.0	PRC
$G_{22}(s)$	9.712	198.0	1.5	PRC

3.3 Matlab fitting

The **tfest** function was used in Matlab to fit the data to a transfer function. The following steps were followed to achieve this:

Data Preparation:

The recorded data was sliced to include a sufficient amount of zero inputs, this is required by the tfest function to properly fit the model. After which the data was broken into a voltage vector (the PWM input), and two temperature vectors (temperatures of each of the transistors). The ambient temperature was subtracted from the temperature vectors and the data was packaged into an iddata object. The table below showcases the temperatures that were subtracted from each graph.

Table 8: Recorded Ambient Temperatures Subtracted for Data Normalization

Dataset	Sensor 1 Ambient ($T_{1,0}$)	Sensor 2 Ambient ($T_{2,0}$)
PWM 1 Step (Estimation)	22.3 °C	21.2 °C
PWM 2 Step (Estimation)	20.3 °C	19.1 °C
PWM 1 Step (Validation)	21.2 °C	20.2 °C
PWM 2 Step (Validation)	21.1 °C	20.2 °C

Model Fitting: Three parameters were passed to the tfest function, namely: the iddata object, the number of poles the system has, and the number of zeros the system has. The number of poles a system has is determined by the number of energy storing elements, in the case of this system the transistors are storing energy in the form of heat, thus the system has two poles. It is common practise to take the number of zeros of an unknown system as: Zeros = Poles -1 . The fitted models were then plotted using the compare function as can be seen in the figures below.

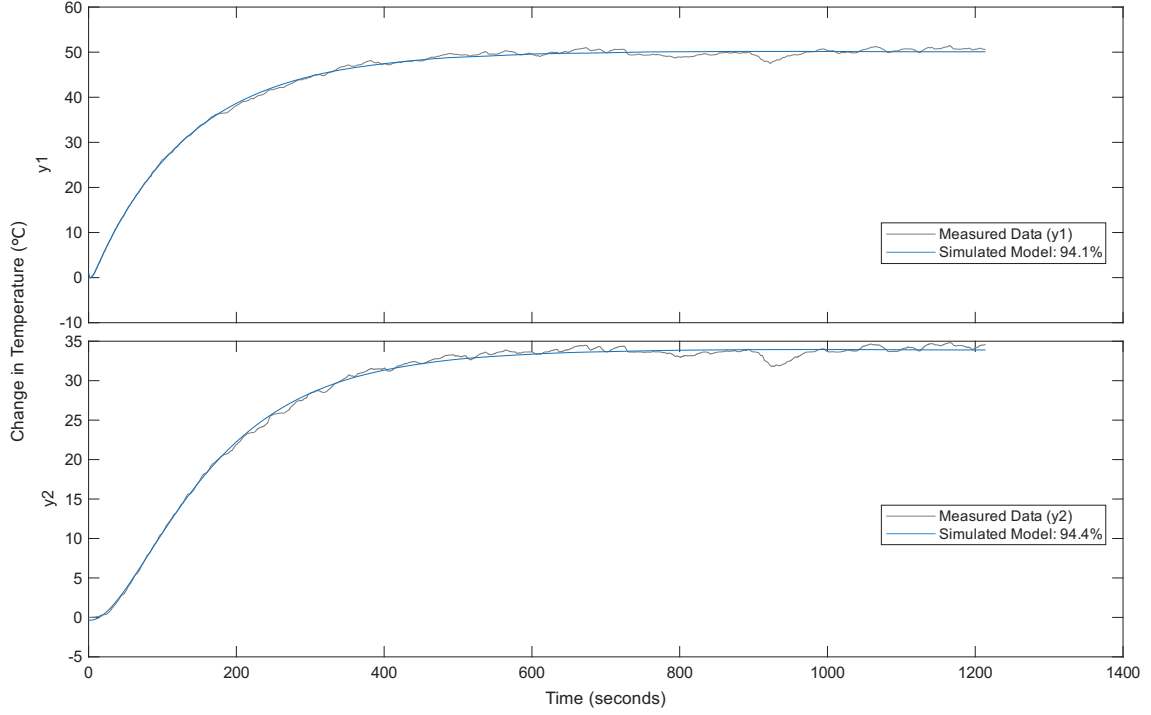


Figure 7: Comparison of the physical sensor data against the simulated SIMO transfer function model for PWM 1.

Figure 7 displays the simulated responses for both G_{11} (top) and G_{21} (bottom) overlaid against the measured physical data, yielding fits of 94.1% and 94.4%, respectively. G_{11} reaches steady state temperature change of approximately 50°C , while G_{21} settles at approximately 33°C . Both simulated responses converge to the same steady-state values as their measured data counterparts.

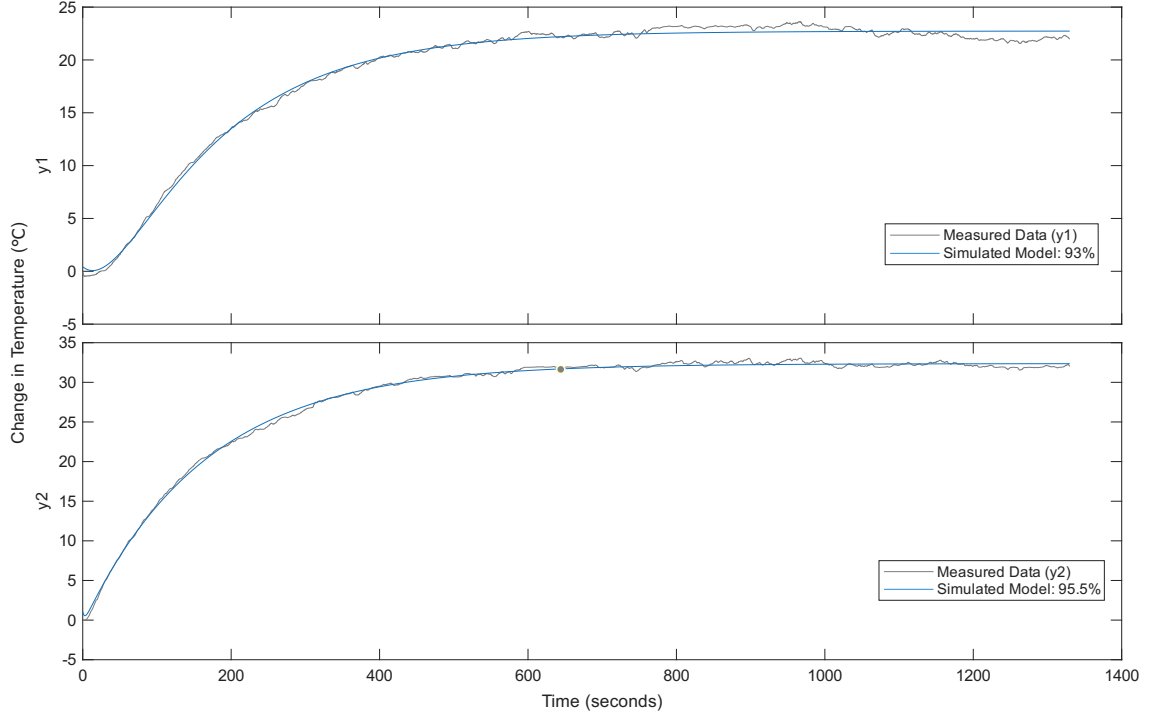


Figure 8: Comparison of the physical sensor data against the simulated SIMO transfer function model for PWM 2.

Figure 8 displays the physical data for a step input on PWM 2, overlaid against the simulated responses for both G_{12} (top) and G_{22} (bottom). The simulated response for G_{22} reaches a steady-state temperature change of approximately 32°C and yields a fit of 95.5% whereas G_{12} yields a fit of 93% and settles at approximately $\Delta 23^{\circ}\text{C}$. Notably, G_{12} settles at a slightly higher temperature change compared to the matching physical data, while G_{22} converges to the exact steady-state value of its measured counterpart.

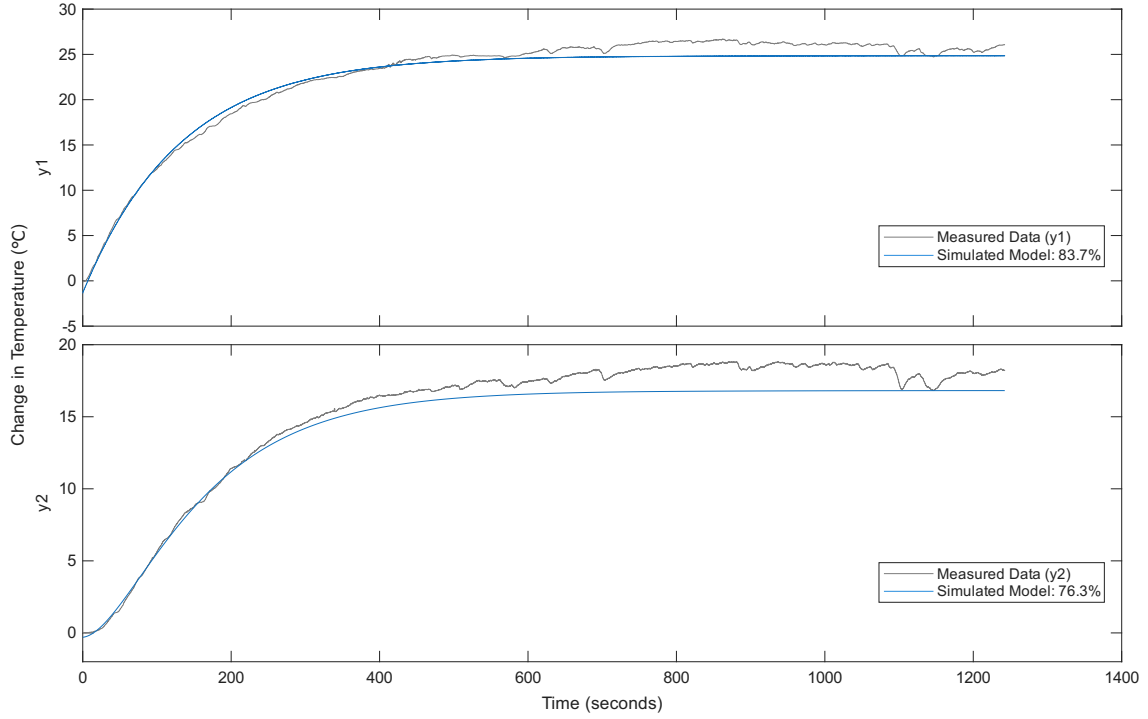


Figure 9: Comparison of the physical sensor data for a 50% duty cycle test case against the simulated SIMO transfer function model for PWM 1.

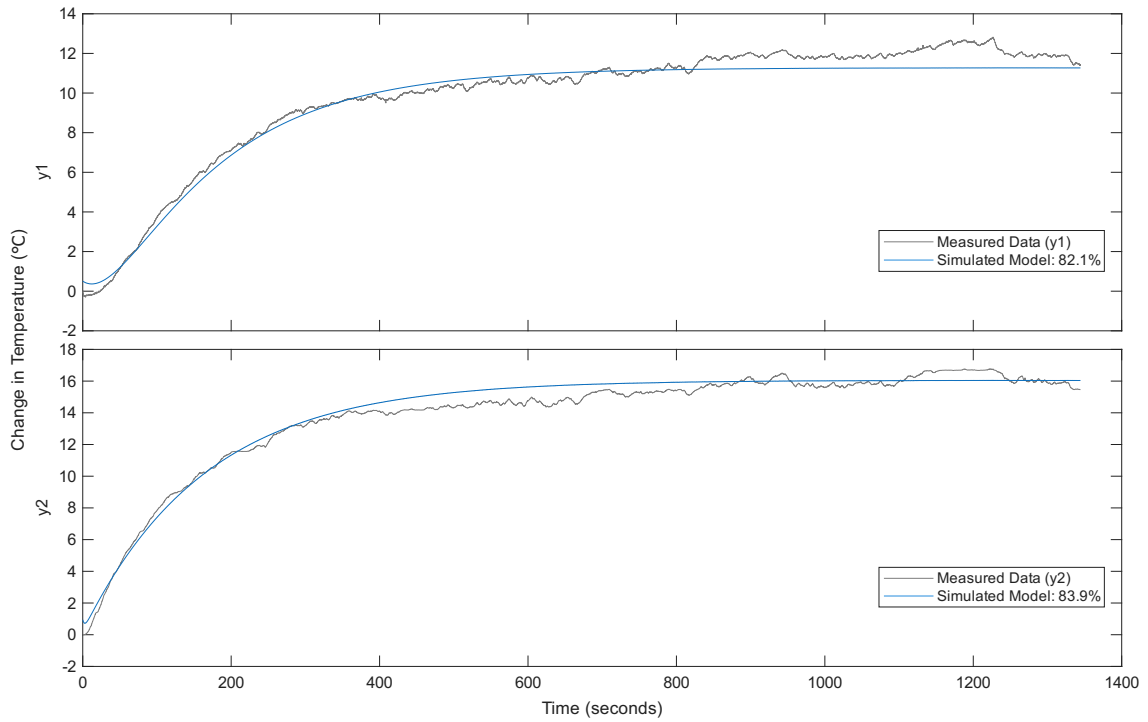


Figure 10: Comparison of the physical sensor data for a 50% duty cycle test case against the simulated SIMO transfer function model for PWM 2.

Figures 9 and 10 display the same simulated models as in Figure 7 and Figure 8 respectively; however, a different validation dataset was used. The duty cycle for both

PWM1 and PWM2 was dropped to 50%. The simulated responses of G_{11} and G_{21} (Figure 9) track the measured data to steady-state amplitudes of $\Delta 24^{\circ}\text{C}$ and $\Delta 16^{\circ}\text{C}$, yielding fits of 83.7% and 76.3% respectively. Both simulated models converge to a lower steady-state value when compared to their measured counterparts. Figure 10 contains the simulated responses of G_{12} (top) and G_{22} (bottom). These responses converge to approximately the same value as their measured counterparts ($\Delta 11^{\circ}\text{C}$ for G_{12} and $\Delta 16^{\circ}\text{C}$ for G_{22}) and yielded fits of 82.1% and 83.9% respectively.

3.3.1 Summary of Validation Data

Table 9 compares the NRMSE fit percentages between the MATLAB and PRC models using the validation dataset.

Table 9: Comparison of NRMSE Fit Percentages: MATLAB vs. Process Reaction Curve (Validation Data)

Transfer Function	MATLAB tfest (%)	PRC (%)	Difference (%)
G_{11} (PWM 1 $\rightarrow$ T1)	83.7	78.0	+5.7
G_{21} (PWM 1 $\rightarrow$ T2)	76.3	29.4	+46.9
G_{12} (PWM 2 $\rightarrow$ T1)	82.1	73.8	+8.3
G_{22} (PWM 2 $\rightarrow$ T2)	83.9	79.7	+4.2

4 Discussion

4.1 Model validation

4.1.1 Accuracy metric

The built-in **compare** function in MATLAB was used to determine the model accuracy. This function utilises the normalised root mean square error (NRMSE) fitness metric. [3] 1 NRMSE was selected mainly due to the advantage it has when comparing different models with different scales.[4] In this system G_{11} converges to approximately $\Delta 50^{\circ}\text{C}$ whereas the simulated response for G_{12} using the validation data set converges to approximately $\Delta 11^{\circ}\text{C}$. To accurately validate the model the scales need to be normalised since a change of 2°C for an amplitude of 11°C is a bigger error than it is for an amplitude of 50°C .

4.1.2 Model accuracy and Transfer function comparisons

Comparing the validation responses in Figures 9 and 10 to the Process Reaction Curve (PRC) models in Figures 5 and 6, it is clear that the MATLAB transfer functions are much more accurate. The **tfest** models consistently resulted in higher NRMSE fit percentages for both the primary and cross-talk heating pathways.

This difference makes sense based on how each model is calculated. MATLAB uses numerical optimization to find the best mathematical fit by looking at the entire dataset at once. On the other hand, the PRC method relies on drawing a single tangent line at the steepest part of the measured curve. Because of this, the PRC method is very sensitive to measurement noise and human error.

Finally, the PRC method forces the model into a basic First-Order Plus Dead Time (FOPDT) structure. The MATLAB models, however, were set up using two poles and one zero. This higher-order structure gave MATLAB the mathematical flexibility it needed to accurately track the initial curves and changing dynamics of the thermal system, which a simple first-order model just ignores.

References

- [1] A. R. Heinz Mittermaier, *Ebb 320 practical 1 guide*, 2026.
- [2] Microchip Technology Inc., *MCP9700/9700A/9701/9701A Low-Power Linear Active Thermistor ICs*, DS20001942, Microchip Technology Inc., 2010. [Online]. Available: <https://ww1.microchip.com/downloads/aemDocuments/documents/MSLD/ProductDocuments/DataSheets/MCP970X-Family-Data-Sheet-DS20001942.pdf>
- [3] The MathWorks Inc., *System identification toolbox user's guide*, Accessed: [Insert Date], The MathWorks Inc., Natick, MA, USA, 2023. [Online]. Available: <https://www.mathworks.com/help/ident/ref/compare.html>
- [4] H. Khoshvaght, R. R. Permala, A. Razmjou, and M. Khiadani, "A critical review on selecting performance evaluation metrics for supervised machine learning models in wastewater quality prediction," *Journal of Environmental Chemical Engineering*, vol. 13, no. 6, p. 119675, 2025. DOI: 10.1016/j.jece.2025.119675

5 Code

5.1 Matlab Code

Listing 1: App Code For Sampling

```
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%% Auto Generated Code Start
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

classdef EBBPrac < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                matlab.ui.Figure
        GridLayout              matlab.ui.container.
            GridLayout
        LeftPanel                matlab.ui.container.
            Panel
        TabGroup                 matlab.ui.container.
            TabGroup
        Tab                      matlab.ui.container.
            Tab
        DataSaveEditField       matlab.ui.control.
            EditField
        DataSaveEditFieldLabel  matlab.ui.control.
            Label
        SamplingtoofastLabel    matlab.ui.control.
            Label
        UpdateChannelsButton     matlab.ui.control.
            Button
        StartButton              matlab.ui.control.
            StateButton
        StopButton               matlab.ui.control.
            StateButton
        OutputDropDown           matlab.ui.control.
            DropDown
        OutputDropDownLabel     matlab.ui.control.
            Label
        DutyCycleEditField       matlab.ui.control.
            NumericEditField
        DutyCycleEditFieldLabel  matlab.ui.control.
            Label
        DisturbanceEditField     matlab.ui.control.
```

```

        NumericEditField
DisturbanceEditFieldLabel    matlab.ui.control.
        Label
SetpointEditField            matlab.ui.control.
        NumericEditField
SetpointEditFieldLabel      matlab.ui.control.
        Label
DEditField                   matlab.ui.control.
        NumericEditField
DEditFieldLabel              matlab.ui.control.
        Label
IEditField                   matlab.ui.control.
        NumericEditField
IEditFieldLabel              matlab.ui.control.
        Label
PEditField                   matlab.ui.control.
        NumericEditField
PEditFieldLabel              matlab.ui.control.
        Label
Tab2                          matlab.ui.container.
        Tab
BaudRateEditField            matlab.ui.control.
        NumericEditField
BaudRateEditFieldLabel      matlab.ui.control.
        Label
COMPortEditField             matlab.ui.control.
        EditField
COMPortEditFieldLabel        matlab.ui.control.
        Label
RightPanel                   matlab.ui.container.
        Panel
UIAxes_2                     matlab.ui.control.
        UIAxes
UIAxes                       matlab.ui.control.
        UIAxes
end

% Properties that correspond to apps with auto-reflow
properties (Access = private)
    onePanelWidth = 576;
end
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Auto Generated Code End
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

properties (Access = private)
    %Live lines 1 and 2 are for the temperature
    measurements.
    liveLine1
    liveLine2
    %Live lines 3 and 4 are for the PWM signals.
    liveLine3
    liveLine4
    P
    I
    D
    setpoint
    disturbance
    duty_cycle
    output
    capturing
    COM_port
    baud_rate
    window_size
    esp32
    data_save_name % Description
end

% Callbacks that handle component events
methods (Access = private)

    % Value changed function: PEditField
    function PEditFieldValueChanged(app, event)
        app.P = app.PEditField.Value;
    end

    % Value changed function: IEditField
    function IEditFieldValueChanged(app, event)
        app.I = app.IEditField.Value;
    end

    % Value changed function: DEditField
    function DEditFieldValueChanged(app, event)
        app.D = app.DEditField.Value;
    end

    % Value changed function: SetpointEditField
    function SetpointEditFieldValueChanged(app, event
    )
        app.setpoint = app.SetpointEditField.Value;

```

```

end

% Value changed function: DisturbanceEditField
function DisturbanceEditFieldValueChanged(app,
    event)
    app.disturbance = app.DisturbanceEditField.
        Value;
end

% Value changed function: DutyCycleEditField
function DutyCycleEditFieldValueChanged(app,
    event)
    app.duty_cycle = app.DutyCycleEditField.Value
        ;
end

% Value changed function: OutputDropDown
function OutputDropDownValueChanged(app, event)
    app.output = str2double(app.OutputDropDown.
        Value);
end

% Value changed function: StartButton
function StartButtonValueChanged(app, event)
    app.StartButton.Visible = "off";
    app.StopButton.Visible = "on";
    app.UpdateChannelsButton.Visible = "on";
    app.capturing = true;

    %Call all of the other functions to make sure
        their values are
    %captured.
    app.PEditFieldValueChanged();
    app.IEditFieldValueChanged();
    app.DEditFieldValueChanged();
    app.SetpointEditFieldValueChanged();
    app.DisturbanceEditFieldValueChanged();
    app.DutyCycleEditFieldValueChanged();
    app.OutputDropDownValueChanged();
    app.BaudRateEditFieldValueChanged();
    app.COMPortEditFieldValueChanged();
    app.DataSaveEditFieldValueChanged();

    if isempty(app.esp32)
        app.esp32 = serialport(app.COM_port, app.
            baud_rate);
        configureTerminator(app.esp32, "LF");
    end
end

```

```

flush(app.esp32);

pause(1);

%Make sure all info is read into esp.
writeline(app.esp32, sprintf("%d, %.2f", 0,
    app.P)); %
writeline(app.esp32, sprintf("%d, %.2f", 1,
    app.I)); %
writeline(app.esp32, sprintf("%d, %.2f", 2,
    app.D)); %
writeline(app.esp32, sprintf("%d, %.2f", 3,
    app.setpoint)); %
writeline(app.esp32, sprintf("%d, %.2f", 4,
    app.disturbance)); %
writeline(app.esp32, sprintf("%d, %d", 5, app
    .duty_cycle)); %
writeline(app.esp32, sprintf("%d, %.2f", 6,
    app.output)); %
writeline(app.esp32, sprintf("%d, %.2f", 7,
    app.capturing)); %

%Clear axes.
cla(app.UIAxes);
cla(app.UIAxes_2);

%Plotting axes for the temperature
measurements.
app.liveLine1 = animatedline(app.UIAxes, '
    Color', 'r');
app.liveLine2 = animatedline(app.UIAxes, '
    Color', 'b');

%Plotting axes for the PWM signals.
app.liveLine3 = animatedline(app.UIAxes_2, '
    Color', 'r');
app.liveLine4 = animatedline(app.UIAxes_2, '
    Color', 'b');

app.window_size = 30; %seconds

%Define data csv that we will save to.
fid = fopen(app.data_save_name + ".csv", 'wt'
    );

while(app.capturing)
    if app.esp32.NumBytesAvailable > 0
        rawData = readline(app.esp32);

```

```

        data = str2double(split(rawData, ','));
        data(5) = data(5)/1000;

        if ~isnan(data(1)) && (length(data)
            == 6)
            %Plot temperature data.
            addpoints(app.liveLine1, data(5),
                data(1));
            addpoints(app.liveLine2, data(5),
                data(2));
            xlim(app.UIAxes, [data(5) - app.
                window_size, data(5)]);

            %Plot PWM signals.
            addpoints(app.liveLine3, data(5),
                data(3));
            addpoints(app.liveLine4, data(5),
                data(4));
            xlim(app.UIAxes_2, [data(5) - app
                .window_size, data(5)]);

            %Write data to file.
            fprintf(fid, rawData);

            if data(end) == 0
                app.SamplingtoofastLabel.
                    Visible = "on";
            else
                app.SamplingtoofastLabel.
                    Visible = "off";
            end
        end

        drawnow limitrate
    end
end

%Close file we are saving to.
fclose(fid);

%Send stop signal.
writeline(app.esp32, sprintf("%d, %.2f", 7,
    app.capturing));

clear app.esp32;
end

```

```

% Value changed function: COMPortEditField
function COMPortEditFieldValueChanged(app, event)
    app.COM_port = app.COMPortEditField.Value;
end

% Value changed function: BaudRateEditField
function BaudRateEditFieldValueChanged(app, event
    )
    app.baud_rate = app.BaudRateEditField.Value;
end

% Value changed function: StopButton
function StopButtonValueChanged(app, event)
    app.capturing = false;
    app.StopButton.Visible = "off";
    app.UpdateChannelsButton.Visible = "off";
    app.StartButton.Visible = "on";

end

% Button pushed function: UpdateChannelsButton
function UpdateChannelsButtonPushed(app, event)
    app.DutyCycleEditFieldValueChanged();
    app.OutputDropDownValueChanged();
    writeline(app.esp32, sprintf("%d, %.2f", 5,
        app.duty_cycle));
    writeline(app.esp32, sprintf("%d, %.2f", 6,
        app.output));
end

% Value changed function: DataSaveEditField
function DataSaveEditFieldValueChanged(app, event
    )
    app.data_save_name = app.DataSaveEditField.
        Value;

end

% Changes arrangement of the app based on
    UIFigure width
function updateAppLayout(app, event)
    currentFigureWidth = app.UIFigure.Position(3)
        ;
    if(currentFigureWidth <= app.onePanelWidth)
        % Change to a 2x1 grid
        app.GridLayout.RowHeight = {449, 449};
        app.GridLayout.ColumnWidth = {'1x'};
        app.RightPanel.Layout.Row = 2;
    end
end

```

```

        app.RightPanel.Layout.Column = 1;
    else
        % Change to a 1x2 grid
        app.GridLayout.RowHeight = {'1x'};
        app.GridLayout.ColumnWidth = {204, '1x'};
        app.RightPanel.Layout.Row = 1;
        app.RightPanel.Layout.Column = 2;
    end
end
end

%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Auto Generated Code Start
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Component initialization
methods (Access = private)

    % Create UIFigure and components
    function createComponents(app)

        % Create UIFigure and hide until all
        % components are created
        app.UIFigure = uifigure('Visible', 'off');
        app.UIFigure.AutoResizeChildren = 'off';
        app.UIFigure.Position = [100 100 702 449];
        app.UIFigure.Name = 'MATLAB App';
        app.UIFigure.SizeChangedFcn =
            createCallbackFcn(app, @updateAppLayout,
                true);

        % Create GridLayout
        app.GridLayout = uigridlayout(app.UIFigure);
        app.GridLayout.ColumnWidth = {204, '1x'};
        app.GridLayout.RowHeight = {'1x'};
        app.GridLayout.ColumnSpacing = 0;
        app.GridLayout.RowSpacing = 0;
        app.GridLayout.Padding = [0 0 0 0];
        app.GridLayout.Scrollable = 'on';

        % Create LeftPanel
        app.LeftPanel = uipanel(app.GridLayout);
        app.LeftPanel.Layout.Row = 1;

```

```

app.LeftPanel.Layout.Column = 1;

% Create TabGroup
app.TabGroup = uitabgroup(app.LeftPanel);
app.TabGroup.Position = [3 3 197 442];

% Create Tab
app.Tab = uitab(app.TabGroup);
app.Tab.Title = 'Tab';

% Create PEditFieldLabel
app.PEditFieldLabel = uilabel(app.Tab);
app.PEditFieldLabel.HorizontalAlignment = '
    right';
app.PEditFieldLabel.Position = [46 385 25
    22];
app.PEditFieldLabel.Text = 'P';

% Create PEditField
app.PEditField = uieditfield(app.Tab, '
    numeric');
app.PEditField.ValueChangedFcn =
    createCallbackFcn(app, @
        PEditFieldValueChanged, true);
app.PEditField.Position = [85 385 100 22];

% Create IEditFieldLabel
app.IEditFieldLabel = uilabel(app.Tab);
app.IEditFieldLabel.HorizontalAlignment = '
    right';
app.IEditFieldLabel.Position = [46 355 25
    22];
app.IEditFieldLabel.Text = 'I';

% Create IEditField
app.IEditField = uieditfield(app.Tab, '
    numeric');
app.IEditField.ValueChangedFcn =
    createCallbackFcn(app, @
        IEditFieldValueChanged, true);
app.IEditField.Position = [85 355 100 22];

% Create DEditFieldLabel
app.DEditFieldLabel = uilabel(app.Tab);
app.DEditFieldLabel.HorizontalAlignment = '
    right';
app.DEditFieldLabel.Position = [46 326 25
    22];

```

```

app.DEditFieldLabel.Text = 'D';

% Create DEditField
app.DEditField = uieditfield(app.Tab, '
    numeric');
app.DEditField.ValueChangedFcn =
    createCallbackFcn(app, @
        DEditFieldValueChanged, true);
app.DEditField.Position = [85 326 100 22];

% Create SetpointEditFieldLabel
app.SetpointEditFieldLabel = uilabel(app.Tab)
    ;
app.SetpointEditFieldLabel.
    HorizontalAlignment = 'right';
app.SetpointEditFieldLabel.Position = [21 296
    49 22];
app.SetpointEditFieldLabel.Text = 'Setpoint';

% Create SetpointEditField
app.SetpointEditField = uieditfield(app.Tab,
    'numeric');
app.SetpointEditField.ValueChangedFcn =
    createCallbackFcn(app, @
        SetpointEditFieldValueChanged, true);
app.SetpointEditField.Position = [85 296 100
    22];

% Create DisturbanceEditFieldLabel
app.DisturbanceEditFieldLabel = uilabel(app.
    Tab);
app.DisturbanceEditFieldLabel.
    HorizontalAlignment = 'right';
app.DisturbanceEditFieldLabel.Position = [1
    265 69 22];
app.DisturbanceEditFieldLabel.Text = '
    Disturbance';

% Create DisturbanceEditField
app.DisturbanceEditField = uieditfield(app.
    Tab, 'numeric');
app.DisturbanceEditField.ValueChangedFcn =
    createCallbackFcn(app, @
        DisturbanceEditFieldValueChanged, true);
app.DisturbanceEditField.Position = [85 265
    100 22];

% Create DutyCycleEditFieldLabel

```

```

app.DutyCycleEditFieldLabel = uilabel(app.Tab
    );
app.DutyCycleEditFieldLabel.
    HorizontalAlignment = 'right';
app.DutyCycleEditFieldLabel.Position = [9 235
    63 22];
app.DutyCycleEditFieldLabel.Text = 'Duty
    Cycle';

% Create DutyCycleEditField
app.DutyCycleEditField = uieditfield(app.Tab,
    'numeric');
app.DutyCycleEditField.ValueChangedFcn =
    createCallbackFcn(app, @
        DutyCycleEditFieldValueChanged, true);
app.DutyCycleEditField.Position = [86 235 100
    22];
app.DutyCycleEditField.Value = 100;

% Create OutputDropDownLabel
app.OutputDropDownLabel = uilabel(app.Tab);
app.OutputDropDownLabel.HorizontalAlignment =
    'right';
app.OutputDropDownLabel.Position = [30 199 41
    22];
app.OutputDropDownLabel.Text = 'Output';

% Create OutputDropDown
app.OutputDropDown = uidropdown(app.Tab);
app.OutputDropDown.Items = {'None', 'Channel1
    ', 'Channel2', 'Both'};
app.OutputDropDown.ItemsData = {'3', '0', '1'
    , '2'};
app.OutputDropDown.ValueChangedFcn =
    createCallbackFcn(app, @
        OutputDropDownValueChanged, true);
app.OutputDropDown.Position = [85 199 100
    22];
app.OutputDropDown.Value = '3';

% Create StopButton
app.StopButton = uibutton(app.Tab, 'state');
app.StopButton.ValueChangedFcn =
    createCallbackFcn(app, @
        StopButtonValueChanged, true);
app.StopButton.Visible = 'off';
app.StopButton.Text = 'Stop';
app.StopButton.Position = [85 168 100 23];

```

```

% Create StartButton
app.StartButton = uibutton(app.Tab, 'state');
app.StartButton.ValueChangedFcn =
    createCallbackFcn(app, @
        StartButtonValueChanged, true);
app.StartButton.Text = 'Start';
app.StartButton.Position = [86 168 100 23];

% Create UpdateChannelsButton
app.UpdateChannelsButton = uibutton(app.Tab,
    'push');
app.UpdateChannelsButton.ButtonPushedFcn =
    createCallbackFcn(app, @
        UpdateChannelsButtonPushed, true);
app.UpdateChannelsButton.Visible = 'off';
app.UpdateChannelsButton.Position = [81 136
    108 23];
app.UpdateChannelsButton.Text = 'Update
    Channels';

% Create SamplingtoofastLabel
app.SamplingtoofastLabel = uilabel(app.Tab);
app.SamplingtoofastLabel.Visible = 'off';
app.SamplingtoofastLabel.Position = [90 12
    101 22];
app.SamplingtoofastLabel.Text = 'Sampling too
    fast!';

% Create DataSaveEditFieldLabel
app.DataSaveEditFieldLabel = uilabel(app.Tab)
;
app.DataSaveEditFieldLabel.
    HorizontalAlignment = 'right';
app.DataSaveEditFieldLabel.Position = [10 96
    61 22];
app.DataSaveEditFieldLabel.Text = 'Data Save'
;

% Create DataSaveEditField
app.DataSaveEditField = uieditfield(app.Tab,
    'text');
app.DataSaveEditField.ValueChangedFcn =
    createCallbackFcn(app, @
        DataSaveEditFieldValueChanged, true);
app.DataSaveEditField.Position = [86 96 100
    22];
app.DataSaveEditField.Value = 'PWM1StepV0';

```

```

% Create Tab2
app.Tab2 = uitab(app.TabGroup);
app.Tab2.Title = 'Tab2';

% Create COMPortEditFieldLabel
app.COMPortEditFieldLabel = uilabel(app.Tab2);
;
app.COMPortEditFieldLabel.HorizontalAlignment = 'right';
app.COMPortEditFieldLabel.Position = [13 385 58 22];
app.COMPortEditFieldLabel.Text = 'COM Port';

% Create COMPortEditField
app.COMPortEditField = uieditfield(app.Tab2, 'text');
app.COMPortEditField.ValueChangedFcn = createCallbackFcn(app, @COMPortEditFieldValueChanged, true);
app.COMPortEditField.Position = [85 385 100 22];
app.COMPortEditField.Value = '/dev/ttyUSB0';

% Create BaudRateEditFieldLabel
app.BaudRateEditFieldLabel = uilabel(app.Tab2);
;
app.BaudRateEditFieldLabel.HorizontalAlignment = 'right';
app.BaudRateEditFieldLabel.Position = [9 355 62 22];
app.BaudRateEditFieldLabel.Text = 'Baud Rate';
;

% Create BaudRateEditField
app.BaudRateEditField = uieditfield(app.Tab2, 'numeric');
app.BaudRateEditField.ValueChangedFcn = createCallbackFcn(app, @BaudRateEditFieldValueChanged, true);
app.BaudRateEditField.Position = [85 355 100 22];
app.BaudRateEditField.Value = 115200;

% Create RightPanel
app.RightPanel = uipanel(app.GridLayout);
app.RightPanel.Layout.Row = 1;
app.RightPanel.Layout.Column = 2;

```

```

        % Create UIAxes
        app.UIAxes = uiaxes(app.RightPanel);
        title(app.UIAxes, 'Title')
        xlabel(app.UIAxes, 'X')
        ylabel(app.UIAxes, 'Y')
        zlabel(app.UIAxes, 'Z')
        app.UIAxes.Position = [7 213 489 235];

        % Create UIAxes_2
        app.UIAxes_2 = uiaxes(app.RightPanel);
        title(app.UIAxes_2, 'Title')
        xlabel(app.UIAxes_2, 'X')
        ylabel(app.UIAxes_2, 'Y')
        zlabel(app.UIAxes_2, 'Z')
        app.UIAxes_2.Position = [8 7 488 207];

        % Show the figure after all components are
        % created
        app.UIFigure.Visible = 'on';
    end
end

% App creation and deletion
methods (Access = public)

    % Construct app
    function app = EBBPrac

        % Create UIFigure and components
        createComponents(app)

        % Register the app with App Designer
        registerApp(app, app.UIFigure)

        if nargin == 0
            clear app
        end
    end

    % Code that executes before app deletion
    function delete(app)

        % Delete UIFigure when app is deleted
        delete(app.UIFigure)
    end
end
end
end

```

```
%
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Auto Generated Code end
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%
```

5.2 Model Fit Code

Listing 2: Model fitting and plotting code

```
% Obtaining the transfer function
% Package the input arrays of both transistors into one
  matrix (Needs to be columns)
PWM1StepV1Data = readmatrix('..\Data1\Step_PWM1_V0.csv');
PWM1StepV1 = PWM1StepV1Data(7920:end,1:5);
PWM2StepV0Data = readmatrix('..\Data1\Step_PWM2_V0.csv');
PWM2StepV0 = PWM2StepV0Data(6400:end,1:5);

PWM1TestDataFile = readmatrix('..\Data1\
  Step_PWM1_VtestData50.csv');
PWM1TestData = PWM1TestDataFile(6230:end,1:5);
PWM2TestDataFile = readmatrix('..\Data1\
  Step_PWM2_VtestData50.csv');
PWM2TestData = PWM2TestDataFile(6160:end,1:5);

% pwm1 data prep
V_PWM1 = [PWM1StepV1(:,3)];
Temp_PWM1 = [PWM1StepV1(:,1)-22.28, PWM1StepV1(:,2)
  -21.24];
time_step = 0.005;

% pwm2 data prep
V_PWM2 = [PWM2StepV0(:,4)];
Temp_PWM2 = [PWM2StepV0(:,1)-20.26, PWM2StepV0(:,2)
  -19.05];

% pwm1 test data prep
V_PWM1_Test = [PWM1TestData(:,3)];
Temp_PWM1_test = [PWM1TestData(:,1)-21.2, PWM1TestData
 (:,2)-20.2];

% pwm2 test data prep
V_PWM2_Test = [PWM2TestData(:,4)];
Temp_PWM2_test = [PWM2TestData(:,1)-21.1, PWM2TestData
 (:,2)-20.2] ;
```

```

% use iddata to set up the data for tfest
tfest_data1 = iddata(Temp_PWM1, V_PWM1, time_step);
tfest_data2 = iddata(Temp_PWM2, V_PWM2, time_step);
testData1 = iddata(Temp_PWM1_test, V_PWM1_Test, time_step
);
testData2 = iddata(Temp_PWM2_test, V_PWM2_Test, time_step
);

% estimate nr of poles and zeros
n_poles = 1;
n_zeros = 0;

% Estimate the transfer functions independently (SISO) to
    avoid idss error
% tfest_data(samples, output, input)
sys11 = tfest(tfest_data1(:, 1, 1), n_poles, n_zeros, NaN
); % PWM1 to T1
sys21 = tfest(tfest_data1(:, 2, 1), n_poles, n_zeros, NaN
); % PWM1 to T2

sys12 = tfest(tfest_data2(:, 1, 1), n_poles, n_zeros, NaN
); % PWM2 to T1
sys22 = tfest(tfest_data2(:, 2, 1), n_poles, n_zeros, NaN
); % PWM2 to T2

% Combine the individual SISO models back into SIMO
    systems
simo_transfer1 = [sys11; sys21];
simo_transfer2 = [sys12; sys22];

% set up graph details
tfest_data1.Name = 'Measured Data';
tfest_data2.Name = 'Measured Data';
testData1.Name = 'Measured Data';
testData2.Name = 'Measured Data';
simo_transfer2.Name = 'Simulated Model';
simo_transfer1.Name = 'Simulated Model';

figure(1)
compare(tfest_data1, simo_transfer1);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

figure(2)
compare(tfest_data2, simo_transfer2);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

```

```

figure(3)
compare(testData1, simo_transfer1);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

figure(4)
compare(testData2, simo_transfer2);
legend('Location', 'best');
ylabel('Change in Temperature (C)')

lines = findobj(gcf, 'Type', 'Line');
set(lines, 'LineWidth', 1.5);

% display the transfer functions of PWM1
[num1, den1] = tfdata(simo_transfer1(1, 1), 'v');
delay1 = simo_transfer1(1, 1).IODelay; % Extract dead
    time
[num2, den2] = tfdata(simo_transfer1(2, 1), 'v');
delay2 = simo_transfer1(2, 1).IODelay;

disp('--- Transfer Function 1 (PWM1 to T1) ---');
disp('Numerator:'); disp(num1);
disp('Denominator:'); disp(den1);
disp(['Estimated Dead Time (s): ', num2str(delay1)]);

disp('--- Transfer Function 2 (PWM1 to T2) ---');
disp('Numerator:'); disp(num2);
disp('Denominator:'); disp(den2);
disp(['Estimated Dead Time (s): ', num2str(delay2)]);

% display transfer functions of PWM2
[num12, den12] = tfdata(simo_transfer2(1, 1), 'v');
delay12 = simo_transfer2(1, 1).IODelay;
[num22, den22] = tfdata(simo_transfer2(2, 1), 'v');
delay22 = simo_transfer2(2, 1).IODelay;

disp('--- Transfer Function 1 (PWM2 to T1) ---');
disp('Numerator:'); disp(num12);
disp('Denominator:'); disp(den12);
disp(['Estimated Dead Time (s): ', num2str(delay12)]);

disp('--- Transfer Function 2 (PWM2 to T2) ---');
disp('Numerator:'); disp(num22);
disp('Denominator:'); disp(den22);
disp(['Estimated Dead Time (s): ', num2str(delay22)]);

```

5.3 MCU Code

```
#include <fmt/core.h>
```

```

#include <Arduino.h>
#include <iostream>
#include <sstream>
#include <vector>
#include <string>
#include <queue>

double P = 0;
double I = 0;
double D = 0;
double setpoint = 0;
double disturbance = 0;
double duty = 128; //50% duty cycle by default.
int control_outputs = 0; //By default only pwm1 is on.
bool start = 0; //Start off.
int message = 100; //Flags for certain messages.
int start_time = 0;
bool startControlling = 0; //See if we should start
    controlling, or continue just measuring.

//PID variables
float error = 0;
float integral = 0;
float previous_error = 0;
float derivative = 0;
bool do_integration = true;

double temp1 = 0;
double temp2 = 0;
int N_samples = 200;
std::queue<double> samples1;
std::queue<double> samples2;

hw_timer_t *timer = NULL;
const int PWM_Out_1 = GPIO_NUM_32;
const int PWM_Out_2 = GPIO_NUM_25;
const int PWM_Meas_1 = GPIO_NUM_33;
const int PWM_Meas_2 = GPIO_NUM_26;

const int Temp_Meas1 = GPIO_NUM_27;
const int Temp_Meas2 = GPIO_NUM_35;

const double A = (3.3/4095.0);
const int PWM_freq = 10;
const int PWM_res = 8;

```

```

const int BaudRate = 1152000;
const int sampling_period = 5000; //In microseconds.
const float dt = sampling_period/1000000.0;
bool first_run = true;

void setup() {
    //Start the USB-C serial/UART port.
    Serial.begin(BaudRate);

    //Setup I/O pins.
    pinMode(Temp_Meas1, INPUT);
    pinMode(Temp_Meas2, INPUT);
    pinMode(PWM_Meas_1, INPUT);
    pinMode(PWM_Meas_2, INPUT);

    ledcChangeFrequency(0, PWM_freq, PWM_res);
    ledcChangeFrequency(1, PWM_freq, PWM_res);
    ledcAttachPin (PWM_Out_1, 0);
    ledcAttachPin(PWM_Out_2, 1);

    timer = timerBegin(0, 80, true);

    samples1.push(0);
    samples2.push(0);
}

void loop() {

    //If receiving communication from computer, update values.
    if(Serial.available() > 0)
    {
        String received = Serial.readStringUntil('\n');
        received.trim();

        std::istringstream stream(received.c_str());
        std::string command;
        std::string value_str;
        double value;

        std::getline(stream, command, ',');
        std::getline(stream, value_str, ',');
        value = std::stod(value_str);

        if(command == "0")
        {
            P = value;
        }
        else if(command == "1")

```

```

{
    I = value;
}
else if(command == "2")
{
    D = value;
}
else if(command == "3")
{
    setpoint = value;
}
else if(command == "4")
{
    disturbance = value;
    disturbance = (pow(2, PWM_res) - 1)*(disturbance
        /100.0);
}
else if(command == "5") //Change from duty cycle set to
    the startControlling flag.
{
    startControlling = value;
}
else if(command == "6")
{
    control_outputs = value;
    //Serial.print("Igot");
    //Serial.println(control_outputs);
}
else if(command == "7")
{
    start = value;

    //Check if start is true, then start timer.
    if(start)
    {
        start_time = millis();
    }
}
}

//If start flag is true, start measuring inputs/start
controlling aswell.
if(start)
{
    //Check when we started in order to see how long we were
    busy.
    timerRestart(timer);
}

```

```

if(startControlling)
{
    //Turn on the appropriate PWM signals.
    ledcWrite(0, duty);
    ledcWrite(1, disturbance);
}

//Interpret sensor data.
double temp1_raw = (analogReadMilliVolts(Temp_Meas1)
    /1000.0 - 0.5)/0.01;
double temp2_raw = (analogReadMilliVolts(Temp_Meas2)
    /1000.0 - 0.5)/0.01;

if(samples1.size() > N_samples)
{
    temp1 -= samples1.front();
    temp2 -= samples2.front();

    samples1.pop();
    samples2.pop();
}

temp1 += temp1_raw/N_samples;
temp2 += temp2_raw/N_samples;

samples1.push(temp1_raw/N_samples);
samples2.push(temp2_raw/N_samples);

//Send sensor input data as a comma seperated list.
std::string sending = fmt::format("{0},{1},{2},{3},{4},{5}",
    temp1, temp2, analogRead(PWM_Meas_1)*A, analogRead(
    PWM_Meas_2)*A, millis() - start_time, message);

Serial.println(String(sending.c_str()));
Serial.flush();

if (startControlling)
{
    //PID controller start
    previous_error = error;
    error = setpoint - temp1;

    // Prevent derivative kick on first run

    if (first_run) {
        previous_error = error;
    }
}

```

```

    first_run = false;
}

float next_integral = integral + error * dt;

derivative = (error - previous_error)/dt;

float pid_out = P * error + I*next_integral + D*
    derivative;
//anti-windup logic
if (pid_out >= 3.3 )
{
    duty = 1;
    if ( error < 0){
        integral = next_integral;
    }
}

else if (pid_out <= 0.0 ) {
    duty = 0.0; // You cannot have a negative duty cycle
               for a simple heater/cooler
    if ( error > 0){
        integral = next_integral;
    }
}

else{
    duty = pid_out/3.3;
    integral = next_integral;
}
duty = (pow(2, PWM_res) - 1)*(duty);

// std::string sending = fmt::format("{0}, {1}, {2},
    {3}, {4}", P, I, D, error, duty);
// Serial.println(String(sending.c_str()));
// Serial.flush();
}
//Otherwise turn off control signals.
else
{
    ledcWrite(0, 0);
    ledcWrite(1, 0);
}

//Check how long everything took in microseconds.
int elapsed = timerReadMicros(timer);

```

```

    if(elapsed <= sampling_period)
    {
        delayMicroseconds(sampling_period - elapsed);
        message = 100; //Reset message flag to show nothing.
    }
    else
    {
        delayMicroseconds(sampling_period);
        message = 0; //Set message flag for sampling too fast.
    }

    //Otherwise deactivate PWM outputs.
}
else
{
    ledcWrite(0, 0);
    ledcWrite(1, 0);
}
}

```

List of Tables

1	MCU Commands	3
2	Comparison of Transfer Functions: Transfer Function Fitting vs. Process Reaction Curve	10
3	Variables for G_{11}	13
4	Variables for G_{21}	13
5	Variables for G_{12}	14
6	Variables for G_{22}	14
7	Process Reaction Curve results summary	15
8	Recorded Ambient Temperatures Subtracted for Data Normalization	15
9	Comparison of NRMSE Fit Percentages: MATLAB vs. Process Reaction Curve (Validation Data)	19

List of Figures

1	Flow Diagram of MCU Code.	4
2	Flow Diagram of Matlab Code.	6
3	Reaction curve for PWM1 step input showing Temp1 and Temp2 responses.	11
4	Reaction curve for PWM2 step input showing Temp1 and Temp2 responses.	11
5	Simulated Process reaction curve model G_{11} (top) and G_{21} (bottom) using a 50% duty cycle as input.	12
6	Simulated Process reaction curve model G_{22} (bottom) and G_{12} (top) using a 50% duty cycle as input.	12
7	Comparison of the physical sensor data against the simulated SIMO transfer function model for PWM 1.	16
8	Comparison of the physical sensor data against the simulated SIMO transfer function model for PWM 2.	17
9	Comparison of the physical sensor data for a 50% duty cycle test case against the simulated SIMO transfer function model for PWM 1. . . .	18
10	Comparison of the physical sensor data for a 50% duty cycle test case against the simulated SIMO transfer function model for PWM 2. . . .	18